an arm in a given posture would pass through a shelf. Inverse kinematics, choosing joint angles that place a gripper at a target pose, is largely geometric. The second is dynamics, meaning how forces produce motion over time. Here the robot must respect joint and torque limits, account for friction at wheels and fingertips, handle collisions and contact, and manage energy, from the battery budget to the momentum of an arm moving near people. Geometry asks whether the robot can be somewhere. Dynamics asks whether it can get there safely and on time.

Most hard decisions in this space mix discrete and continuous choices. Which shelf to visit next is discrete; how hard to accelerate is continuous. Quantum optimization methods, at least in their common forms, fit the discrete part most naturally, which is why promising research tends to carve a combinatorial slice out of a larger physical problem rather than handing the whole thing to a QPU.

Turning a motion problem into something a QPU can read

Both QAOA and annealers usually take problems written as a QUBO, short for quadratic unconstrained binary optimization, which is mathematically equivalent to the Ising model from physics. Every decision becomes a binary variable, and the objective is a sum of terms involving single variables and pairs of variables. A question such as whether task three goes in slot five becomes a yes-or-no variable. A continuous quantity such as a joint angle must be chopped into bins and spelled out across several bits. That is workable only for a bounded, carefully chosen subproblem; a robot's full continuous dynamics, with contact, friction and time-varying forces, does not shrink into a compact QUBO.

Constraints are the awkward part. A QUBO cannot natively say that each slot holds exactly one task, or that a joint must never pass its limit, so violations are added to the objective as penalty terms. Set the penalty weight too low and the solver cheerfully returns cheap answers that break the rules. Set it too high and the penalties swamp the landscape, real cost differences shrink by comparison, and limited-precision hardware struggles to separate good solutions from mediocre ones. Finer discretization makes the model more faithful but demands more qubits and couplings. Every encoding trades fidelity against size and solvability, and none removes the need to check answers against the real constraints afterward.

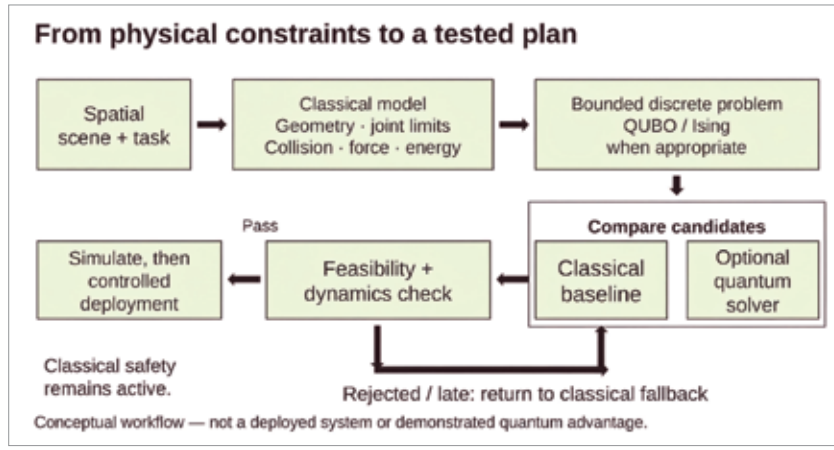


Figure 1: A physical task is modelled classically, a bounded discrete subproblem is encoded where appropriate, a classical baseline and an optional quantum solver are compared, and candidates are checked for feasibility and tested in simulation; rejected or late results fall back to the classical plan

Recent research shows both the appeal and the limits. A 2025 *Scientific Reports* paper proposes a quantum formulation of robot posture, an inverse-kinematics problem; it reads best as an exploration of formulation, not evidence of practical speedup. Novara, Boggio and Volpe go further up the control stack, proposing to recast polynomial nonlinear model predictive control as a QUBO, and state plainly that any benefit depends on an efficient quantum computer. Figure 1 sketches the disciplined path such a subproblem would need to travel before it could influence a real machine.

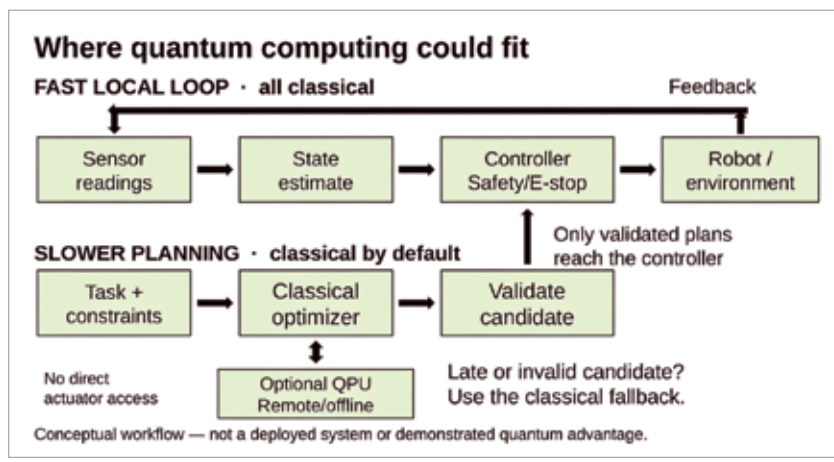


Figure 2: Feedback, safety monitoring and emergency stop stay in a fast, all-classical local loop, while optional quantum planning runs remotely or offline on a slower timescale and reaches the controller only after classical validation